

Foundations for Programming Languages

2: Imperative languages

(structured programming, control flow)

Isao Sasano

Imperative languages

In imperative languages, computations are sequences of actions such as assignments and procedure calls.

Famous imperative languages :

- Fortran (1957, John Backus, American, 1977 Turing award),

- Algol 60 (1960, International committee)

- Pascal (1970, Niklaus Wirth, Swiss, 1984 Turing award),

- C (1972, Dennis Ritchie, American, 1983 Turing award)

Original Fortran does not have loop constructs like while statements, so many goto statements are used. (Fortran 90 (1991) has loop constructs.)

Languages like Algol and Pascal have constructs like while statements and compound statements like begin-end.

Assignments

Basic units of imperative languages are assignments, which change the values of variables.

(Examples)

$x := 2+3$

$x := a[i]$

$a[i] := x$

Procedure calls can indirectly change the values of variables.

(Example)

$\text{read}(x)$

Structured programming

We should design programming languages so that we easily understand what kind of computations are performed when we see the program.

Structured programming: The structure of program text should help us understand what the program does.

Carefully designed structured programs can be just as efficient as unstructured ones.

[Reference] Edsger Dijkstra, “Go to statement considered harmful”, *Communication of the ACM*, Vol. 11, No. 3, pp. 147-148, 1968.

Syntax-directed control flow

Structured Control Flow: A program is structured if the flow of control through the program is evident from the syntactic structure of the program text.

“Evident” is defined as *single entry / single exit*.

Statements specify actions and the flow of control around actions, which we illustrate in Pascal.

Compound statements (**begin-end**)

Conditional statements (**if-then, if-then-else**)

Looping constructs (**while**)

Selection (**case**)

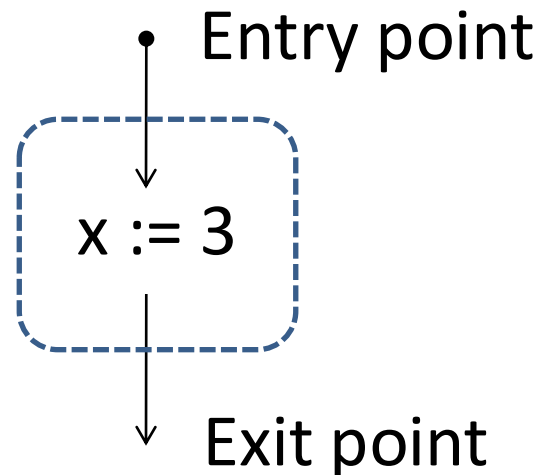
Basic statements --- assignments

Assignments are single entry / single exit.

For example, an assignment statement

$x := 3$

has the following control flow.



Sequences of statements

In Pascal, a sequence of statements is represented by delimiting statements $s_1, s_2, \dots, s_n$ with semicolons.

$s_1; s_2; \dots; s_n$

(ex.) $\text{temp} := x;$

$x := y;$

$y := \text{temp}$

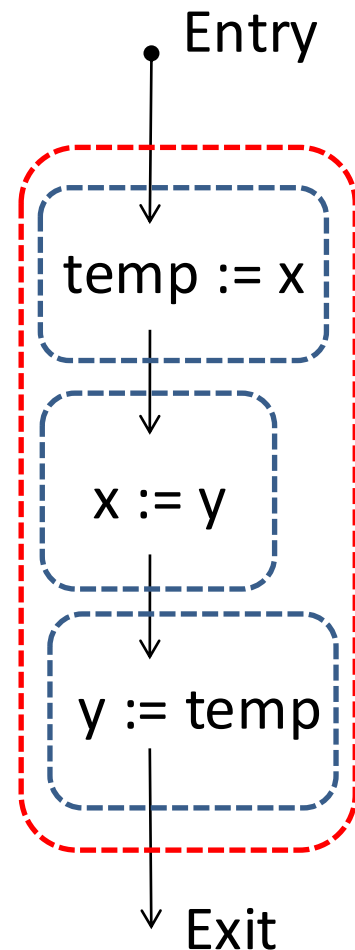
Compound statements (or blocks)

In Pascal, by surrounding a sequence of statements by **begin** and **end**, we get a statement called a *compound statement* or a *block*.

(ex.) **begin** temp := x; x := y; y := temp **end**

We can write a compound statement wherever a statement can appear. The number of statements in a compound statement can be 0, so **begin end** is a compound statement.

A compound statement is single entry / single exit when each of the statements in the compound statement is single entry / single exit.



Conditional statements

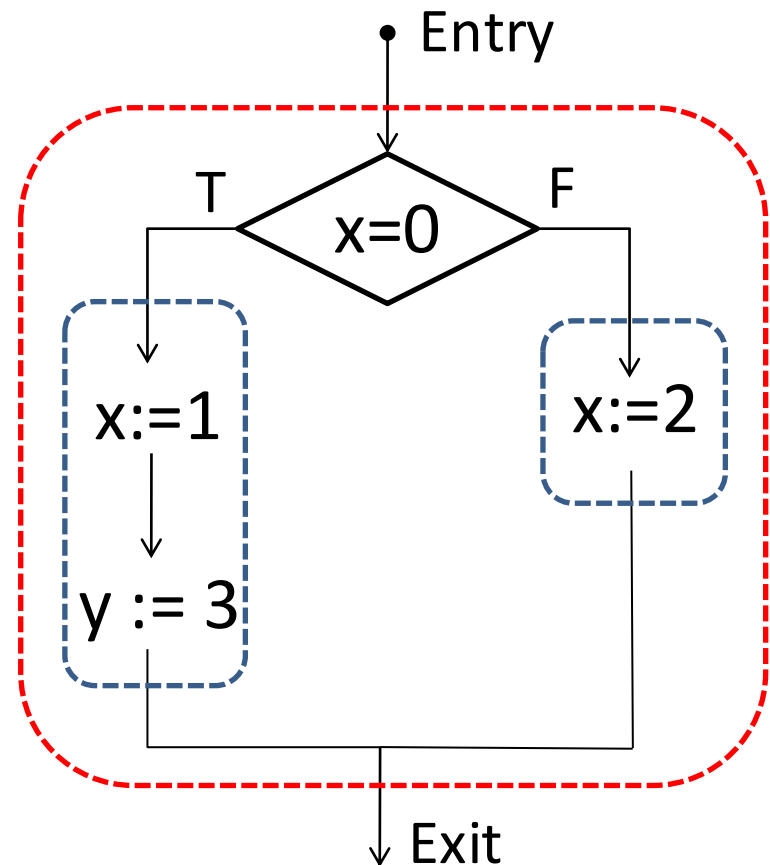
In Pascal, a conditional statement has either of the following form (E is an expression, S (with a subscript) is a statement).

if E then S_1 else S_2

if E then S

(ex.) **if $x=0$ then**
 begin $x:=1$; $y:=3$ end
 else $x:=2$

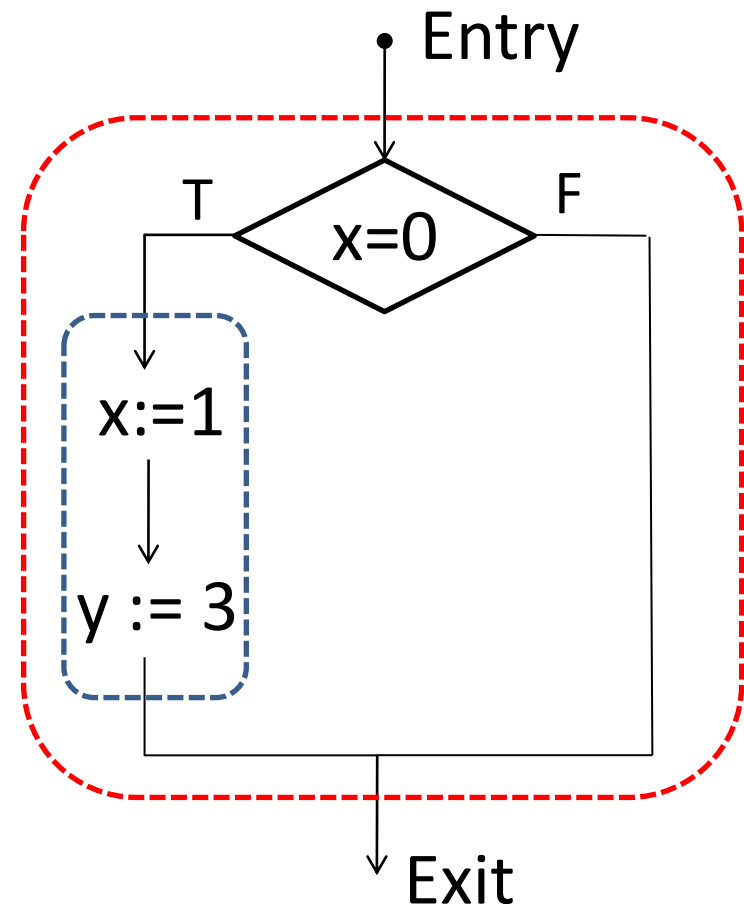
A conditional statement **if E then S_1 else S_2** is single entry / single exit when S_1 and S_2 are single entry and single exit.



if-then statements

(ex.) **if** $x=0$ **then**
 begin
 $x:=1$;
 $y:=3$
 end

A conditional statement **if** E **then** S is single entry / single exit when S is single entry and single exit.



Exercise 1

Illustrate the control flow graph of the following program fragment.

if $x > 0$ **then** $x := x - 1$

else if $y > 0$ **then** $y := y - 1$

else $y := y + 1$

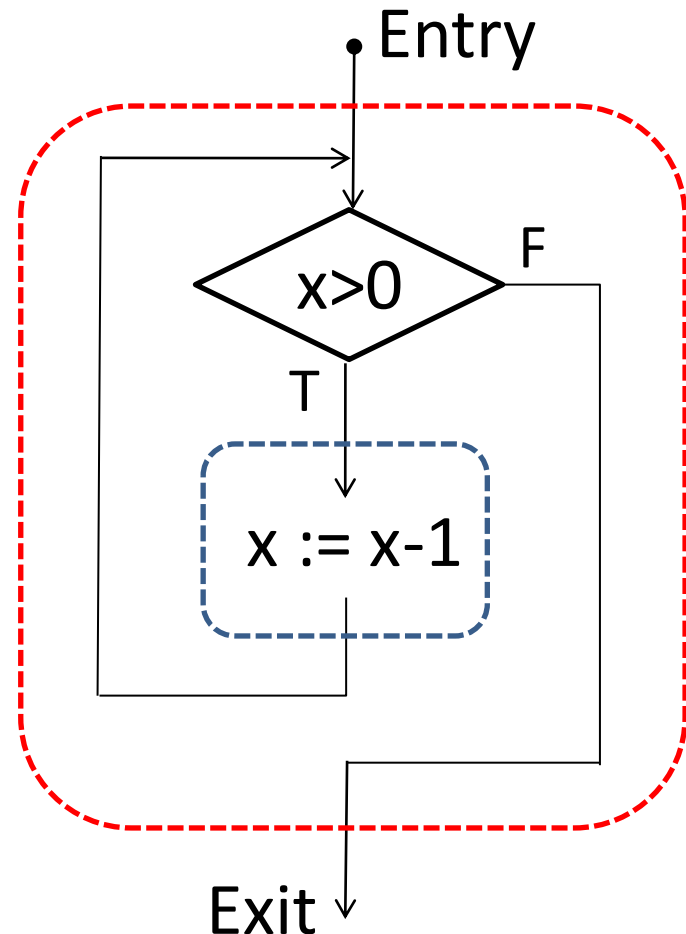
Loop statements (while statements)

In Pascal, a while statement has the following form.

while E **do** S

(ex.) **while** $x > 0$ **do** $x := x-1$

A while statement **while** E **do** S is single entry / single exit if the body S is single entry/ single exit.



Selection

In Pascal, a selection statement has the following form.

case E **of**

$constant_1 : S_1;$

...

$constant_n : S_n$

end

(In C, a selection is a switch statement.)

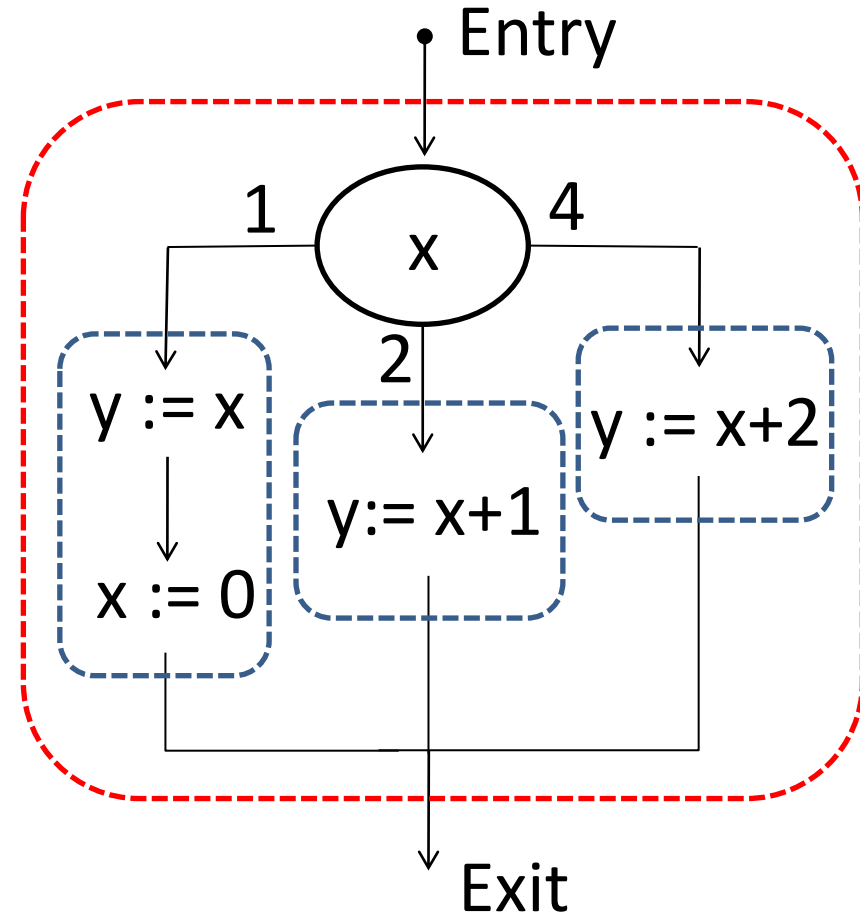
(ex.) **case** x **of**

1 : **begin** $y:=x$; $x:=0$ **end**;

2: $y:=x+1$;

4: $y:=x+2$

end



Exercise 2

Illustrate the control flow graph of the following program fragment.

$y := 3;$

case x **of**

1 : $y := 1;$

2 : $y := x * 2;$

3 : **if** $z = 0$ **then**

$y := y * y$

else

$y := y * y * y$

end

Handling special cases in loops

Break and continue statements (in C)

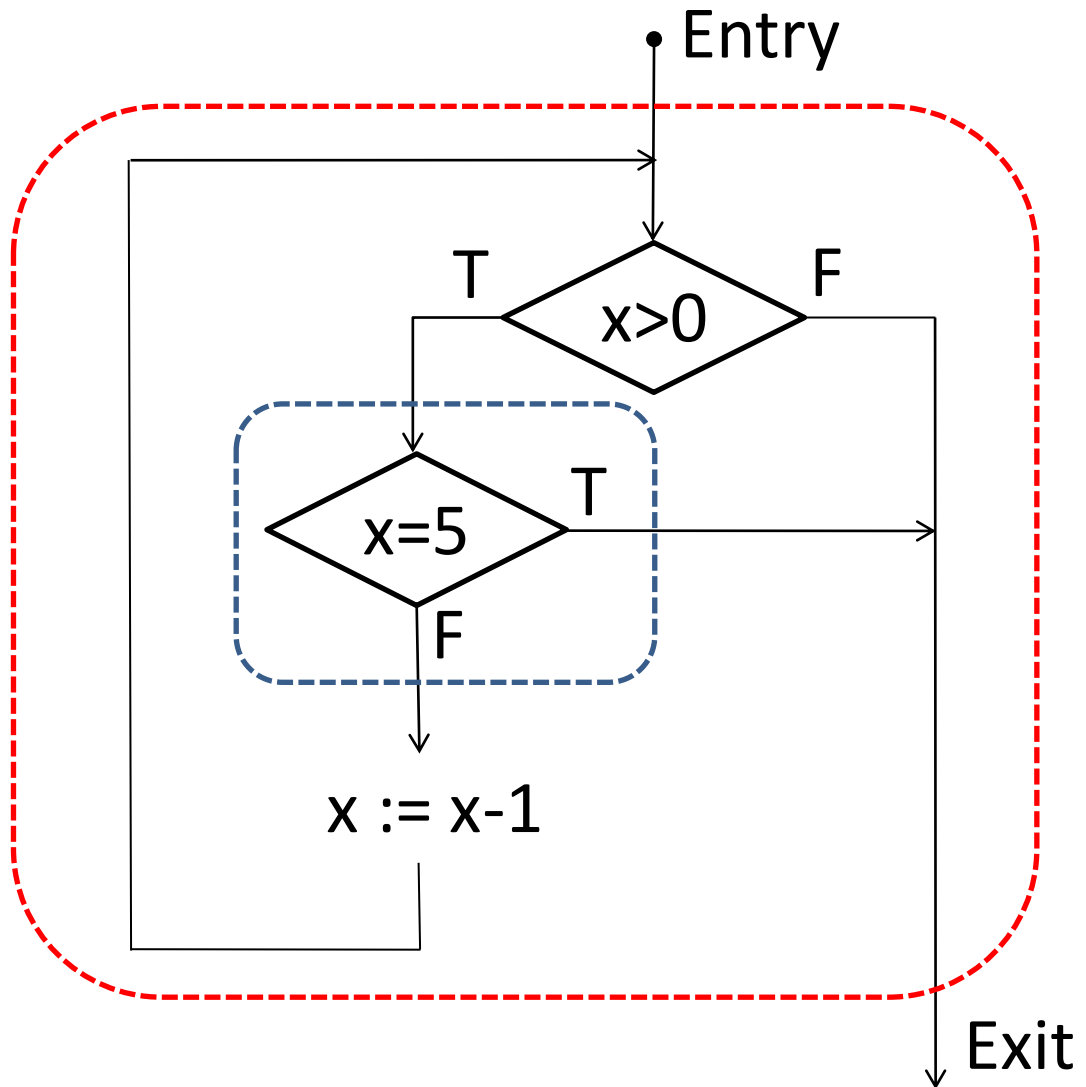
A **break** statement sends control out of the inner-most enclosing loop to the statement following the loop.

A **continue** statement repeats the inner-most enclosing loop by sending control to the beginning of the loop.

An example using a break statement

```
while  $x > 0$  do  
  begin  
    if  $x = 5$  then  
      break;  
     $x := x - 1$   
  end
```

The **if** statement is single entry / two exits but the **while** statement is single entry / single exit.



Exercise 3

Illustrate the control flow graph of the following program fragment.

```
while x>0 do  
  begin  
    if x=3 then  
      break;  
    y := y + 1;  
    x := x - 1  
  end
```

Exercise 4

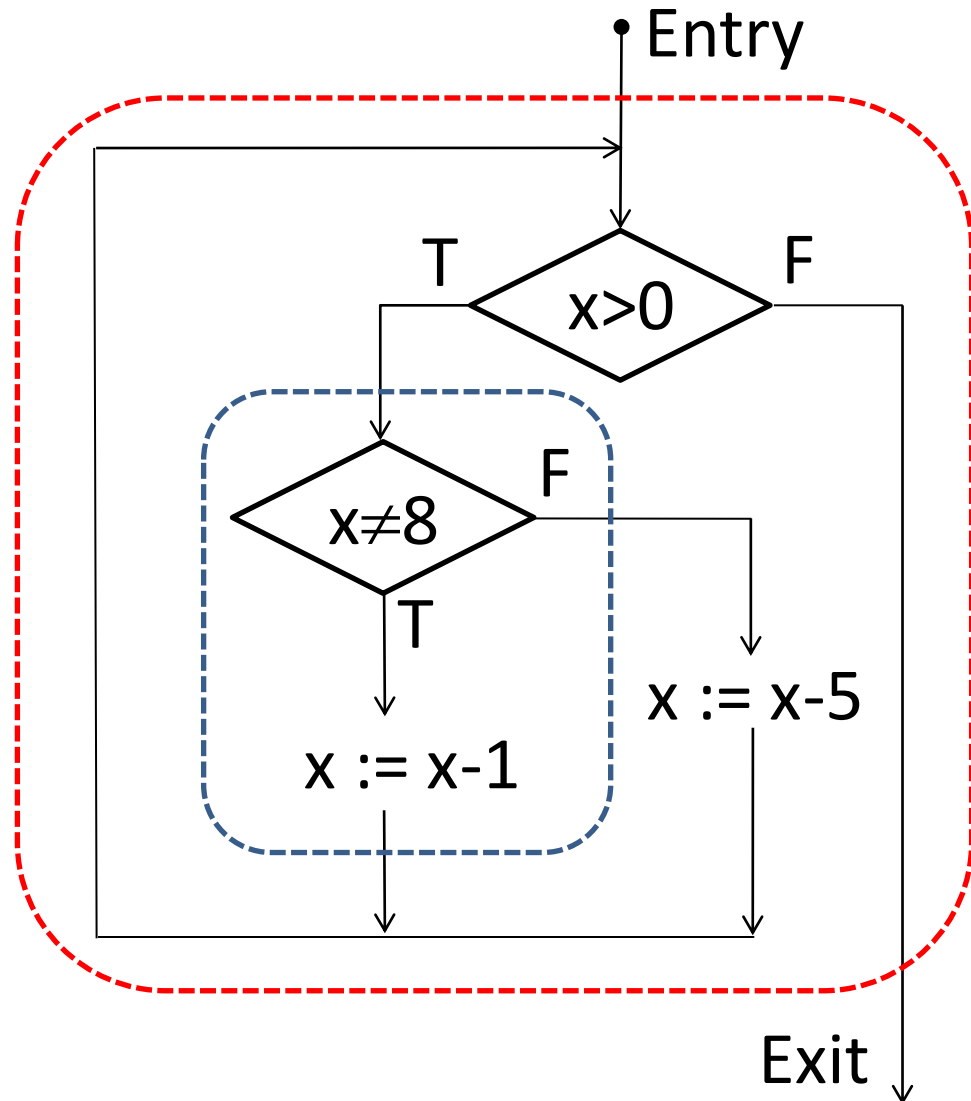
Illustrate the control flow graph of the following program fragment.

```
while x>0 do  
  begin  
    while y>0 do  
      begin  
        if x=3 then  
          break;  
        z := z + 1;  
        y := y - 1  
      end;  
      x := x - 1  
    end  
  end
```

An example using a continue statement

```
while  $x > 0$  do  
  begin  
    if  $x \neq 8$  then  
      begin  
         $x := x - 1$ ;  
        continue  
      end;  
     $x := x - 5$   
  end
```

The **if** statement is single entry/ two exits but the **while** statement is single entry / single exit.



Exercise 5 Illustrate the control flow graph of the following program fragment.

```
while x>0 do  
  begin  
    while y>0 do  
      begin  
        if x≠3 then  
          begin  
            y := y - 1;  
            continue  
          end  
          z := z + 1; y := y - 1  
        end;  
        x := x - 1  
      end  
    end
```

goto statements

goto statements has the following form.

goto *label*

(ex.)

L: $x := x - 4;$

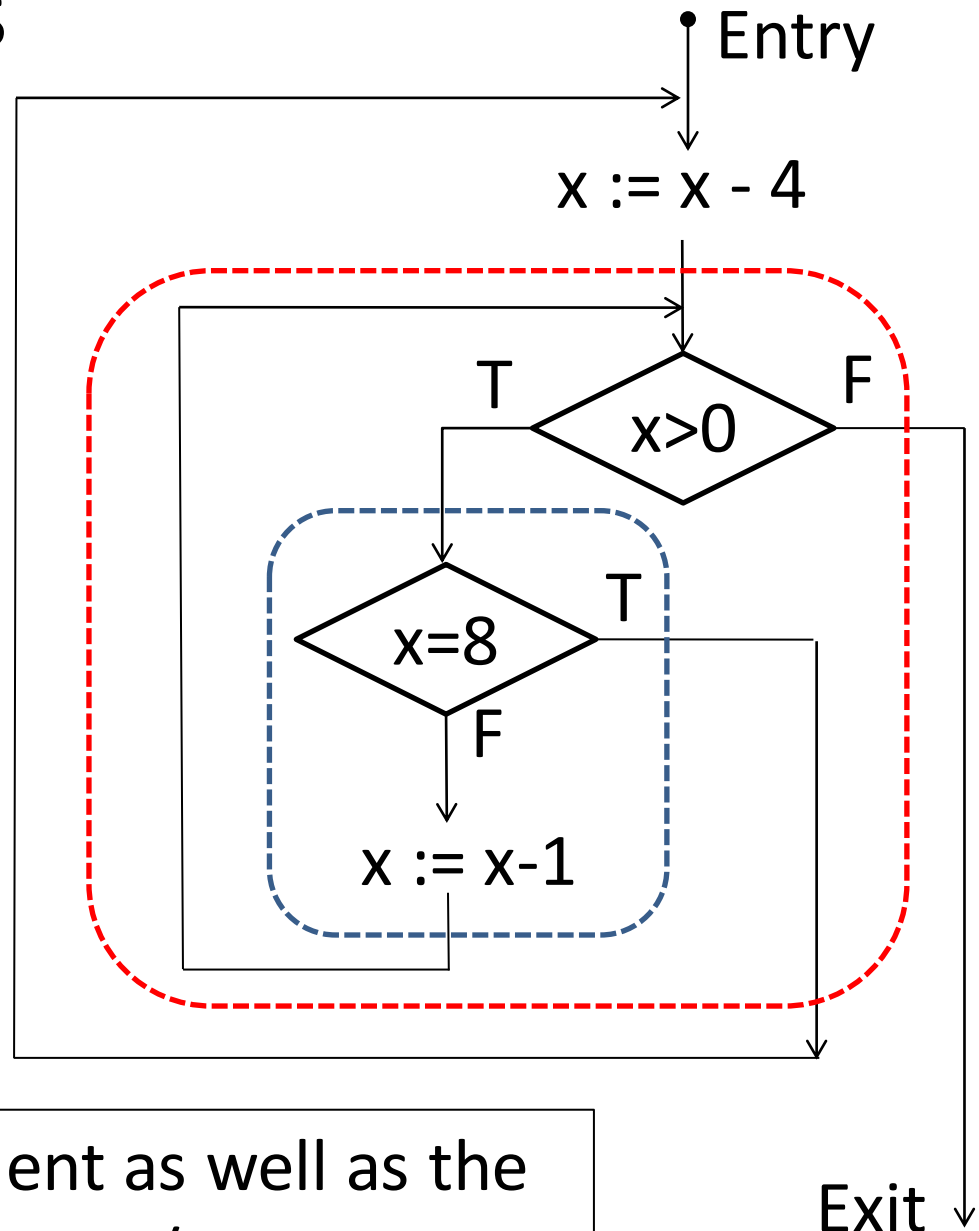
while $x > 0$ **do**

if $x = 8$ **then**

goto L

else

$x := x - 1$



Now the **while** statement as well as the **if** statement is single entry / two exits.

Exercise 6

Illustrate the control flow graph of the following program fragment.

$x := 10;$

$\text{sum} := 0;$

L: $\text{sum} := \text{sum} + x;$

$x := x - 1;$

if $x > 0$ then

 goto L

Exercise 7

How many entries and exits do the two **if** statements have?

```
while  $x > 0$  do  
  begin  
    while  $y > 0$  do  
      begin  
        if  $x = 3$  then  
          break;  
        L:  $z := z + 1$ ;  
         $y := y - 1$   
      end;  
       $x := x - 1$ ;  
      if  $x = 2$  then goto L  
    end
```

return statements

In Modula-2, **return** statements have either of the following forms.

return

return E

(In C, they are **return**; or **return** E ;))

A **return** statement sends control back to the caller (with carrying the value of E).

A **return** statement sends control out of the enclosing procedure (or function), while a **break** statement sends control out of the enclosing loop.